macOS App Deployment & Apple VPP with Microsoft Intune (App Store + Enterprise Apps)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — macOS App Deployment & Apple VPP with Microsoft Intune 4

1. Solution Overview — macOS App Deployment & Apple VPP with Microsoft Intune 4

What this solution is . . . 4

The macOS app types in Intune (know which to use) . . . 5

VPP licensing: device vs. user (choose deliberately) . . . 5

How it works . . . 5

Named objects this solution creates (and reuses) . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 7

2. Requirements — macOS App Deployment & Apple VPP with Microsoft Intune 7

R1. Apple Business Manager (or Apple School Manager) account . . . 7

R2. An ABM location with acquired app licenses . . . 7

R3. VPP token for the location (the ABM ↔ Intune license link) . . . 8

R4. Managed Apple ID / token owner recorded . . . 8

R5. Target Macs enrolled and managed in Intune . . . 8

R6. Entra pilot group for app targeting . . . 8

R7. Signed package(s) for enterprise (LOB) apps . . . 8

R8. Decide the licensing model per app (device vs. user) . . . 9

R9. Network access to Apple app & VPP endpoints . . . 9

R10. macOS enrollment/app deployment allowed in Intune . . . 9

Global Requirements — Intune Solutions (category-level) 9

G1. Microsoft Entra ID tenant . . . 9

G2. Microsoft Intune subscription . . . 9

G3. Administrative roles . . . 10

G4. Device enrollment in Intune . . . 10

G5. Pilot security group . . . 10

G6. Supported Windows baseline . . . 10

G7. Network access to Microsoft cloud endpoints . . . 10

3. Implementation — macOS App Deployment & Apple VPP with Microsoft Intune 10

Phase 1 — Acquire licenses and connect VPP to Intune . . . 11

Phase 2 — Add and assign the App Store (VPP) app . . . 11

Phase 3 — Add and assign an enterprise (LOB) app . . . 12

Phase 4 — Pilot, validate, expand . . . 12

Ongoing — token renewal (do not skip) . . . 13

5. Testing & Validation — macOS App Deployment & Apple VPP with Microsoft Intune 13
Test 1 — Tenant readiness (token + license sync) . . . 13
Test 2 — App Store (VPP) app assigned to the pilot . . . 14
Test 3 — Silent install on a pilot Mac (device licensing) . . . 14
Test 4 — VPP license consumed for the device . . . 14
Test 5 — Enterprise (LOB) app installs . . . 14
Test 6 — Install status reported in Intune . . . 14
Test 7 — License headroom is sufficient . . . 14
Test 8 — Updates flow . . . 14
Acceptance gate . . . 15
6. Troubleshooting — macOS App Deployment & Apple VPP with Microsoft Intune 15
VPP apps don't appear in Intune / licenses show zero . . . 15
Token is expired or about to expire . . . 15
App Store app won't install on the Mac . . . 15
User is prompted for an Apple ID (expected silent) . . . 16
VPP license not consumed for a device . . . 16
Enterprise (LOB) `.pkg`/`.dmg` fails to install . . . 16
LOB update doesn't take . . . 16
Install status stuck "Pending"/"Failed" in Intune . . . 16
Token / sync expiry (preventive) . . . 16
Useful checkpoints . . . 17
7. Best Practices — macOS App Deployment & Apple VPP with Microsoft Intune 17
Licensing model . . . 17
Token & license-sync lifecycle (the #1 cause of outages) . . . 17
App type selection . . . 18
Assignment & targeting hygiene . . . 18
Updates & versioning . . . 18
Operations & documentation . . . 18
Security & posture . . . 18
8. Deliverables Summary — macOS App Deployment & Apple VPP with Microsoft Intune 19
Documentation (this guide) . . . 19
PowerShell scripts (Microsoft Graph) — `MACVPP-Scripts.zip` . . . 19
Hero image . . . 20
How it's delivered . . . 20
Appendix A — PowerShell Scripts 21

Executive Summary — macOS App Deployment & Apple VPP with Microsoft Intune

Getting the **right applications onto managed Macs** — licensed, current, and installed without touching the device — is a persistent gap for many organizations. This solution closes it. It stands up **macOS app deployment through Microsoft Intune with Apple's Volume Purchase Program (VPP)**: you connect the Apple Business Manager **VPP location token**, sync licenses, deploy **App Store** and **enterprise (line-of-business)** apps, and keep them updated — all from the Intune console.

This solution delivers the complete, field-tested method — written so a single administrator can follow it end to end:

• **No-touch, licensed delivery.** Apps acquired in Apple Business Manager install **silently** on

managed Macs via VPP **device licensing** — **no Apple ID, no App Store sign-in**, no hands on the device — and Intune tracks each license consumed and reclaims it when a device retires.

• **Store *and* enterprise, one pane.** Public **App Store** apps and your own **signed** `.pkg`**/**`.dmg`

line-of-business apps deploy, update, and report from the same Intune console.

• **Right licensing model, deliberately chosen.** Device vs. user licensing is decided per app, with

device licensing as the recommended default for company-owned Macs.

• **Renew-aware.** The guide treats the **annually-expiring VPP token** as a scheduled operational

task — with a script that flags it before it lapses — so license sync and installs never silently stop.

The package follows the standard repeatable shape — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with exact, click-by-click Intune and Apple Business Manager steps, a requirements guide that tells you how to verify *and* fix each prerequisite, five ready-to-run PowerShell scripts (Microsoft Graph), and testing/troubleshooting guides with clear pass/fail gates.

Outcome: the right apps on every Mac — App Store and enterprise, licensed via VPP and kept current — delivered by your own team, proven on a named pilot before you scale.

1. Solution Overview — macOS App Deployment & Apple VPP with Microsoft Intune

What this solution is

This solution delivers a repeatable way to get the **right applications onto managed Macs** — licensed, assigned, and kept current — entirely from **Microsoft Intune**, without touching the device. It combines two things:

1. **Apple Volume Purchase Program (VPP)** — the licensing side of **Apple Business Manager (ABM)**.

Your organization acquires app licenses in ABM (free or paid) against a **location**, exports that location's **VPP token** (a server token, `.vpptoken`), and connects it to Intune. Intune then **syncs the licenses** and can install **App Store** apps and assign their licenses automatically.

1. **macOS app deployment in Intune** — the delivery side. Intune installs **App Store (VPP) apps**,

line-of-business (LOB) apps you package yourself (`.pkg` / signed `.dmg`), and **unmanaged PKG** installers, targeting Entra security groups and reporting install and licensing status.

Together they give you licensed, hands-off app delivery to the Mac fleet — deployed pilot-first and rollback-ready.

The macOS app types in Intune (know which to use)

Intune → **Apps** → **macOS** → **Add** offers several app types. This solution uses these:

• **macOS app (App Store — VPP).** A public App Store app whose **licenses come from VPP**. Intune

installs it and consumes a VPP license per install. *Requires a VPP token.* Best for anything on the App Store (e.g. a productivity or reader app).

• **macOS line-of-business app (LOB).** A **signed** `.pkg` you upload to Intune. Intune installs it

as a managed app and can update it by uploading a newer package. Best for in-house or vendor apps not on the App Store.

• **macOS app (DMG).** A vendor-provided **signed** `.dmg` containing a `.app`. Intune installs and

can report/uninstall it. Best for common third-party apps shipped as DMG.

• **macOS app (PKG) — unmanaged.** A **PKG installer** run once on the device (no license tracking,

no automatic uninstall). Best for installers that must run scripts or place files outside `/Applications`. Least managed; use when the managed types don't fit.

> ***Rule of thumb:*** *if it's on the App Store, use* ***VPP****; if you have a* ***signed*** `.pkg`/`.dmg`*, use the matching* ***managed*** *type; only fall back to* ***unmanaged PKG*** *when the others can't.*

VPP licensing: device vs. user (choose deliberately)

When you assign a VPP (App Store) app, Intune offers two license types:

• **Device licensing (recommended default).** The license is bound to the **device**, not to an

Apple ID. The app installs **silently**, the user needs **no Apple ID and no App Store sign-in**, and the license returns to the pool when the device retires. Best for company-owned, managed Macs.

• **User licensing.** The license is bound to a **user's Managed Apple ID / Apple ID**. The user must

be **invited and accept** VPP terms and be signed in to the App Store. More friction; use only when a specific app genuinely requires user-based licensing.

This solution standardizes on **device licensing** for company Macs, with notes for user licensing.

How it works

1. **Acquire licenses in ABM.** In Apple Business Manager, an admin buys (or adds free) app licenses

under a **location**. Each location has its **own** VPP token.

1. **Connect the token to Intune.** You upload the location's `.vpptoken` in Intune as a **VPP

token**. Intune establishes the trust and begins syncing.

1. **Sync licenses.** Intune pulls the purchased apps and license counts from ABM. Synced apps become

available to add as **App Store** apps in Intune.

1. **Add & assign apps.** You add the store app (or upload a LOB/DMG/PKG), pick **device** or **user**

licensing, and assign it **Required** (or **Available**) to a **pilot** Entra group.

1. **Install & license, hands-off.** Managed Macs install the app on the next check-in; Intune

consumes and reports a VPP license per device. Updates flow as new versions publish (store) or as you upload newer packages (LOB).

1. **Renew the token.** VPP tokens **expire (annually)**; a lapsed token stops license sync and new

installs, so renewal is a scheduled operational task.

Named objects this solution creates (and reuses)

To keep the deployment repeatable, this solution creates and **reuses** these named objects:

- **VPP token:** `VPP – Corporate (Location A)` — the connected Apple VPP location token in Intune.
- **Pilot group:** `PILOT – macOS App Deployment` — an Entra security group of pilot devices/users.
- **Store app assignment:** a chosen App Store app (e.g. a reader/productivity app) assigned

Required, **device-licensed**, to the pilot group.

- **Enterprise (LOB) app:** a signed `.pkg` you package, added as a **macOS LOB app** and assigned

Required to the pilot group.

Broader waves and the production rollout **reuse the same objects** (renaming *Pilot* → the wave/all target as you expand).

Benefits

- **No-touch delivery.** Apps install silently on managed Macs — no Apple ID, no App Store sign-in

(device licensing), no hands on the device.

- **Licensed correctly.** VPP tracks each license; Intune reports consumption and reclaims on retire.
- **Store + enterprise, one pane.** App Store apps and your own `.pkg`/`.dmg` deploy and report from

the same Intune console.

- **Kept current.** Store apps update as new versions publish; LOB updates by uploading a new package.
- **Pilot-first, rollback-ready.** Everything is validated on a named pilot before fleet rollout.

Limitations (be honest with buyers)

- **VPP requires Apple Business Manager** and a **location** with acquired licenses; a Mac not

managed by Intune can't receive managed apps.

• **App Store apps require a VPP token**; the token **expires annually** and must be renewed or license sync and new installs stop.

• **Device licensing** needs a **supervised / managed** Mac; **user licensing** requires the user to accept VPP terms and sign in to the App Store.

• **LOB `.pkg`/`.dmg` must be signed** (and, where required, notarized) to install cleanly on modern macOS; **unmanaged PKG** is install-once with no license tracking or managed uninstall.

• **Free apps still consume a VPP "license"** entry in ABM (count is effectively unlimited but the app must be acquired in ABM first).

• Personally-owned (BYO) Macs and per-user App Store purchases outside VPP are **out of scope**.

The standard deployment model

This solution uses **one** model so the process is repeatable across our catalog:

> ***Company-owned, Intune-managed Macs receiving VPP-licensed App Store apps (device-licensed) and signed enterprise LOB apps, assigned to a named pilot group first, then expanded in waves.***

Every step follows the repeatable pattern: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

2. Requirements — macOS App Deployment & Apple VPP with Microsoft Intune

Confirm **every** requirement below before you begin. Each entry follows the same pattern: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites (tenant, Intune licensing, admin roles, pilot group, network) are covered once in the **Global Requirements — Intune Solutions** document; the items here are specific to Apple VPP and macOS app deployment.

R1. Apple Business Manager (or Apple School Manager) account

Description: A verified Apple Business Manager (ABM) organization. **Why:** VPP lives inside ABM; app licenses are acquired there against a location, and the VPP token is exported from there. Without ABM there is no VPP. **Verify:** Sign in to `https://business.apple.com` with an ABM Administrator (or Content Manager) account; the organization status is **active**. **Implement:** Enroll at `https://business.apple.com/enroll` using your **D-U-N-S** number; complete Apple's verification (a phone callback to a verifiable company contact). Allow 1–5 business days.

R2. An ABM location with acquired app licenses

Description: At least one ABM **location**, with the target apps **acquired** (free or paid) under it. This solution names the working location **Location A**. **Why:** VPP licenses are held **per location**, and each location has **its own** token. Intune can only sync and install apps that have been acquired in that location. **Verify:** ABM → **Apps** (or **Content** → **Apps**); confirm the app appears with a license count under your

location. ABM → **Locations** shows the location exists. **Implement:** ABM → **Apps**, search the App Store app, choose the **location**, set a quantity, and **Get** it (free apps: quantity is effectively unlimited but the app must still be acquired). Create a location under ABM → **Locations** if none exists.

R3. VPP token for the location (the ABM ↔ Intune license link)

Description: The location's downloaded **VPP server token** (`.vpptoken`), connected in Intune. **Why:** This token is the trust link that lets Intune sync that location's licenses and install its App Store apps. **Verify:** Intune → **Tenant administration** → **Connectors and tokens** → **Apple VPP tokens** (also reachable via **Apps** → **macOS** → **… VPP**); a token is listed as **Active/Valid** with a future expiration. This solution names it `VPP – Corporate (Location A)`. **Implement:** In ABM → **Preferences** → **Payments and Billings** (or the location's **Apps** area), **Download** the location's VPP token; in Intune, **Add** an Apple VPP token, sign in with the ABM managed Apple ID that owns the token, upload the `.vpptoken`, and give it the name above. **Renew annually.**

R4. Managed Apple ID / token owner recorded

Description: The **managed Apple ID** used to download and own the VPP token, documented. **Why:** The token is tied to that Apple ID; renewals must use the **same** identity, and losing track of it makes renewal painful. **Verify:** Intune → the VPP token shows the **Apple ID** it was uploaded with; you have that Apple ID recorded in your runbook. **Implement:** Record the managed Apple ID (and the ABM location it belongs to) in your operations runbook the first time you connect the token.

R5. Target Macs enrolled and managed in Intune

Description: The Macs that will receive apps are **enrolled** in Intune and manageable. **Why:** Managed apps and VPP **device licensing** only reach enrolled, managed devices. Device licensing further expects a **supervised / company-owned** Mac. **Verify:** Intune → **Devices** → **macOS**; the target device appears, is **managed**, and (for device licensing) shows **Supervised = Yes / Corporate-owned**. **Implement:** Enroll Macs (Automated Device Enrollment or Company Portal). A separate solution covers zero-touch macOS enrollment; company-owned + supervised is required for silent device-licensed VPP.

R6. Entra pilot group for app targeting

Description: A Microsoft Entra security group for the pilot — this solution names it `PILOT – macOS App Deployment`. **Why:** Every app assignment targets a group; piloting to a small, named group first is the model. **Verify:** Entra → **Groups**; the group `PILOT – macOS App Deployment` exists and contains your pilot device(s) or user(s). **Implement:** Entra → **Groups** → **New group** → **Security**, name it exactly as above, add the pilot members. The included `MACVPP-Set-ScopeTargeting.ps1` can create/populate it.

R7. Signed package(s) for enterprise (LOB) apps

Description: For any in-house/vendor app you deploy as LOB, a **signed** `.pkg` (or vendor **signed** `.dmg`); notarized where Apple requires it. **Why:** Modern macOS (Gatekeeper) blocks unsigned/unnotarized installers; Intune's managed `.pkg` type expects a **component/distribution PKG** signed with a Developer ID Installer certificate. **Verify:** On a Mac, `pkgutil --check-signature <pkg>` (or `spctl -a -vv <app>` for a `.dmg`'s app) reports a valid Developer ID signature. **Implement:** Obtain the signed installer

from the vendor, or sign your own with a **Developer ID Installer** certificate (`productsign`) and **notarize** it. For installers that can't be made into a managed `.pkg`, use the **unmanaged PKG** app type (install-once, no license/uninstall management).

R8. Decide the licensing model per app (device vs. user)

Description: For each VPP app, a decision: **device** licensing (default) or **user** licensing. **Why:** It changes the user experience and prerequisites — device licensing is silent and needs no Apple ID; user licensing needs the user invited, accepting VPP terms, and signed in to the App Store. **Verify:** You have, per app, a documented choice and (for user licensing) a plan to invite users. **Implement:** Default to **device** licensing for company Macs. Only choose **user** licensing when an app specifically requires it; then plan the VPP user invitation flow.

R9. Network access to Apple app & VPP endpoints

Description: Outbound HTTPS to Apple's VPP, App Store, and content endpoints. **Why:** License sync, App Store app delivery, and content downloads all depend on Apple endpoints; a block silently stalls installs. **Verify:** From the device network, reach `https://vpp.itunes.apple.com`, `https://buy.itunes.apple.com`, Apple's content/CDN ranges, and the Apple push range `*.push.apple.com` (TCP 443 and 5223). **Implement:** Allow Apple's published network ranges (Apple support article HT210060) plus the Microsoft Intune/Entra endpoints through proxy/firewall. Do **not** TLS-inspect Apple push/VPP.

R10. macOS enrollment/app deployment allowed in Intune

Description: macOS platform is permitted by enrollment restrictions, so managed devices can receive apps. **Why:** A restriction that blocks macOS stops devices from enrolling and thus from receiving apps. **Verify:** Intune → **Devices** → **Enrollment** → **Enrollment device platform restrictions** → **macOS** shows platform **Allowed**. **Implement:** Edit the default (or a targeted) restriction to **Allow** macOS for the pilot group.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant**

status shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation — macOS App Deployment & Apple VPP with Microsoft Intune

Follow these steps **in order**. The *why* for each capability lives in the Solution Overview; the steps below are deliberately short and exact so you can **click to implement**. Throughout, the repeatable pattern applies: **Assess → Configure → Pilot → Validate → Expand.** Do everything against the `PILOT – macOS App Deployment` group first.

All admin-center steps use the Microsoft Intune admin center at `https://intune.microsoft.com` and Apple Business Manager at `https://business.apple.com` unless noted.

Phase 1 — Acquire licenses and connect VPP to Intune

Step 1. Acquire the app licenses in Apple Business Manager

1. In ABM, go to **Apps** (older tenants: **Content** → **Apps**).
2. Search the App Store app you want to deploy (e.g. a reader or productivity app).
3. Select your **location** — this solution uses **Location A**.
4. Set a **quantity** (paid apps) — free apps are effectively unlimited but must still be acquired.
5. Click **Get**. The app now shows a license count under Location A. Repeat for each app.

*Acquire **every** app you intend to deploy under the **same location** so a single token covers them.*

Step 2. Download the location's VPP token

1. In ABM, open **Preferences (your name, bottom-left)** → **Payments and Billings** (or the location's **Apps** area).

1. Under the location, **Download** the **VPP server token** — a `.vpptoken` file. **Record the managed Apple ID** shown as the token owner (you reuse it at every renewal — Requirements R4).

Step 3. Connect the VPP token in Intune

1. In Intune, go to **Tenant administration** → **Connectors and tokens** → **Apple VPP tokens** (or **Apps** → **macOS** → **…** VPP area).

1. Click **Create / Add**.

2. **Basics:** enter the **Apple ID** (the managed Apple ID that owns the token) and **Browse** to the `.vpptoken`.

1. **Settings:** name it `VPP – Corporate (Location A)`; set the **country/region** to match the location; choose the **default license type** to reflect this solution's default — **device**. Accept the terms.

1. **Review + Create.** The token appears as **Active/Valid** with a future expiration.

Step 4. Sync licenses from Apple

1. On the token `VPP – Corporate (Location A)`, click **Sync**.
2. Wait for the sync to complete (Apple rate-limits syncs; allow a few minutes).
3. Confirm the acquired apps now appear in Intune → **Apps** → **macOS** as available App Store apps with

license counts.

Prefer to confirm from the command line? Run `MACVPP-Discovery.ps1` — it reports the token status/expiry and synced app/license counts.

Phase 2 — Add and assign the App Store (VPP) app

Step 5. Add the App Store (VPP) app to Intune

1. Go to **Apps** → **macOS** → **Add**.
2. **App type:** choose **macOS app (App Store)** (the VPP-licensed store app type). Click **Select**.
3. **Search** the App Store and pick the app you acquired in Step 1 (or select it from the synced list). Confirm the correct **bundle ID** and publisher.
1. Complete **App information** (name, publisher, category, logo as desired) → **Next**.

Step 6. Assign the store app to the pilot — device-licensed

1. On the app's **Assignments** tab, under **Required**, click **Add group**.
2. Choose `PILOT – macOS App Deployment`.
3. Set **License type = Device** (this solution's default — silent, no Apple ID needed).
4. Leave uninstall-on-unassign per your policy → **Review + Save**.

> *Use **Available** (instead of Required) if you want the app to appear in Company Portal for on-demand install; keep **Required** for hands-off delivery. For **user** licensing, set **License type = User** and ensure users have accepted VPP terms (Requirements R8).*

Phase 3 — Add and assign an enterprise (LOB) app

Step 7. Add the enterprise app (signed `.pkg`)

1. Go to **Apps** → **macOS** → **Add**.
2. **App type:** choose **macOS line-of-business (LOB) app** for a signed `.pkg` (or **macOS app (DMG) **for a signed** `.dmg`**. Click** Select**.
1. **Upload** the signed package (Requirements R7). For a `.pkg`, provide the app's **bundle ID** and **build/version** and any **child/included app bundle IDs** Intune prompts for (used to detect install). → **Next**.
1. Complete **App information** → **Next**.

> *Can't produce a managed `.pkg`? Use **macOS app (PKG) — unmanaged** instead: install-once, no license tracking, no managed uninstall. Prefer the managed types when possible.*

Step 8. Assign the enterprise app to the pilot

1. On the app's **Assignments** tab, under **Required**, click **Add group**.
2. Choose `PILOT – macOS App Deployment` → **Review + Save**.

> *LOB/DMG/PKG apps are **not** VPP-licensed, so there is no license type to choose — assignment is group-based only.*

Phase 4 — Pilot, validate, expand

Step 9. Trigger and observe install on a pilot Mac

1. On a pilot Mac (a member of `PILOT – macOS App Deployment`), open **Company Portal** or **System Settings** → **General** → **Device Management** and **Sync**, or wait for the next check-in.
1. Confirm the **App Store (VPP)** app installs **silently** (device licensing needs no Apple ID) and the **enterprise** `.pkg` installs to `/Applications`.

Step 10. Validate installation and licensing

1. Run `MACVPP-Validation.ps1 -Serial <serial>` (or `-DeviceName`): confirm the device is **managed macOS**, the **VPP app is installed and licensed to the device**, and the **LOB app is installed**, with a clear PASS/FAIL verdict.
1. In Intune, cross-check **Apps** → **(the app)** → **Device install status** = **Installed**, and **Apps** → **(the VPP app)** → **(VPP licenses)** shows a license **consumed** for the device.

Step 11. Confirm license consumption and headroom

1. Run `MACVPP-Report.ps1` for a tenant-wide view: per-app **total vs. consumed** VPP licenses, install counts, and any app **out of licenses**.
1. Confirm enough license headroom before widening (acquire more in ABM if needed, then **Sync**).

Step 12. Expand in waves

1. Once the pilot passes, broaden targeting: add production groups to the app **Required** assignments (or retarget the pilot group to your wave/all-devices group) using `MACVPP-Set-ScopeTargeting.ps1` (defaults to **-WhatIf** — preview first, then apply).
1. Keep **device** licensing as the default; monitor consumption with `MACVPP-Report.ps1` each wave.
2. Roll out in waves, validating each wave with `MACVPP-Validation.ps1` before widening.

Ongoing — token renewal (do not skip)

The **VPP token expires annually**. A lapsed token **stops license sync and new installs**. Before expiry: in ABM download a **fresh** `.vpptoken` for **Location A** (same managed Apple ID), then in Intune open `VPP – Corporate (Location A)` → **Renew** and upload it. Run `MACVPP-Discovery.ps1` regularly — it flags the token when it is within 30 days of expiry.

5. Testing & Validation — macOS App Deployment & Apple VPP with Microsoft Intune

Validate against a **pilot device** before expanding. Each test has an explicit **pass/fail gate**. Use the included PowerShell scripts (Microsoft Graph) for tenant-side checks and the on-device steps for the user experience.

Test 1 — Tenant readiness (token + license sync)

Procedure: Run `MACVPP-Discovery.ps1`. **Pass:** The VPP token `VPP – Corporate (Location A)` is **Active/Valid** (expiry > 30 days), **last sync is recent**, and at least one **App Store app with synced licenses** is present. **Fail:** Token missing/expiring within 30 days, or no synced apps → resolve before assigning apps.

Test 2 — App Store (VPP) app assigned to the pilot

Procedure: In Intune → **Apps** → **macOS** → **(the store app)** → **Assignments**; confirm the assignment. **Pass:** The app is **Required** to `PILOT – macOS App Deployment` with **License type = Device**. **Fail:** Wrong group, wrong license type, or not assigned → correct the assignment (Implementation Step 6).

Test 3 — Silent install on a pilot Mac (device licensing)

Procedure: Sync a pilot Mac and wait for delivery. **Pass:** The VPP app installs **silently** with **no Apple ID prompt and no App Store sign-in**, and appears in `/Applications`. **Fail:** The user is prompted for an Apple ID → the assignment is user-licensed, or the Mac is not supervised/company-owned (Requirements R5, R8). Switch to device licensing and re-check.

Test 4 — VPP license consumed for the device

Procedure: In Intune → **Apps** → **(the VPP app)** → **(VPP licenses/associated licenses)**, or run `MACVPP-Validation.ps1 -Serial <serial>`. **Pass:** A VPP license shows **consumed/assigned** to the pilot device; consumed count increased by one. **Fail:** No license consumed → confirm licenses synced (Test 1), the app is device-licensed (Test 2), and the device checked in.

Test 5 — Enterprise (LOB) app installs

Procedure: Confirm the signed `.pkg`/`.dmg` app on the pilot Mac (or `MACVPP-Validation.ps1`). **Pass:** The LOB app is present in `/Applications` and Intune reports **Device install status = Installed** for it. **Fail:** Not installed → verify the package is **signed/notarized** (Requirements R7) and assigned to the pilot group (Step 8); check `Console.app`/Intune install error for the reason.

Test 6 — Install status reported in Intune

Procedure: In Intune → **Apps** → **(each app)** → **Device install status**. **Pass:** The pilot device shows **Installed** for both the VPP app and the LOB app. **Fail:** Shows **Failed/Pending** past a check-in cycle → see Troubleshooting; re-run `MACVPP-Validation.ps1` after fixing.

Test 7 — License headroom is sufficient

Procedure: Run `MACVPP-Report.ps1`. **Pass:** Every deployed app has **consumed < total** licenses (headroom for the next wave); no app is **out of licenses**. **Fail:** An app at 100% consumption → acquire more licenses in ABM under Location A, then **Sync** before expanding.

Test 8 — Updates flow

Procedure: For a store app, confirm a newer App Store version updates on managed Macs; for LOB, upload a newer signed `.pkg` (higher version) and confirm the update installs. **Pass:** Managed Macs move

to the newer version without user interaction (store) / after the package update (LOB). **Fail:** Version doesn't advance → confirm the app is **Required**, the device checks in, and (LOB) the new package version/build is **higher** than the installed one.

Acceptance gate

Proceed to **Expand** only when Tests 1–8 pass on at least one pilot Mac — the App Store app installs **silently and device-licensed**, the enterprise app installs, both report **Installed**, and license headroom is confirmed.

6. Troubleshooting — macOS App Deployment & Apple VPP with Microsoft Intune

Work top-down: confirm the **token & license sync** first, then **assignment**, then the **device** (install), then **licensing**. Most failures are an expired/unsynced VPP token, a wrong license type, an unsigned package, or a Mac that isn't supervised/managed.

VPP apps don't appear in Intune / licenses show zero

- **Token not synced.** Intune → `VPP – Corporate (Location A)` → **Sync**; allow a few minutes (Apple

rate-limits). Confirm with `MACVPP-Discovery.ps1`.

- **App not acquired in ABM.** The app must be **Get**-acquired under **Location A** first

(Requirements R2), then synced.

- **Wrong location/token.** Licenses are per location — make sure the app was acquired in the same

location whose token you connected (R2/R3).

Token is expired or about to expire

- **Renew, don't recreate.** In ABM download a **fresh** `.vpptoken` for Location A (same managed

Apple ID), then Intune → token → **Renew** and upload. A lapse **stops sync and new installs**.

- Run `MACVPP-Discovery.ps1` — it flags a token within 30 days of expiry.

App Store app won't install on the Mac

- **Device not managed/checked in.** Intune → **Devices** → **macOS** must show the device; **Sync** it

(Requirements R5).

- **Out of licenses.** `MACVPP-Report.ps1` — if consumed = total, acquire more in ABM under Location A

and **Sync** (Requirements R2).

- **Assignment missing/wrong group.** Confirm the app is **Required** to **`PILOT – macOS App

Deployment`** (Implementation Step 6).

- **Network blocks Apple.** Ensure `vpp.itunes.apple.com` / App Store / content endpoints are reachable

and **not** TLS-inspected (Requirements R9).

User is prompted for an Apple ID (expected silent)

- **The app is user-licensed, not device-licensed.** Set **License type = Device** on the assignment (Step 6) so it installs silently.
- **Mac not supervised/company-owned.** Device licensing expects a supervised, corporate Mac (Requirements R5). Fix ownership/supervision, or accept user licensing and invite the user.

VPP license not consumed for a device

- **Licenses not synced** (Test 1) or **app not device-licensed** (Test 2) — correct and re-check.
- **Device hasn't checked in** — **Sync** the device and re-run `MACVPP-Validation.ps1`.

Enterprise (LOB) `.pkg`/`.dmg` fails to install

- **Package not signed/notarized.** Gatekeeper blocks it — verify with `pkgutil --check-signature` (Requirements R7); re-sign with a **Developer ID Installer** cert and notarize.
- **Wrong/missing bundle ID or version** on the LOB upload — Intune can't detect install; re-enter the correct **bundle ID** and **build/version** (Implementation Step 7).
- **Not a distribution/component PKG** — some raw payloads don't install as managed; if it can't be made into a managed `.pkg`, use the **unmanaged PKG** type (Step 7 note).

LOB update doesn't take

- **Version/build not higher.** Intune only updates when the new package's version/build **exceeds** the installed one — bump it and re-upload (Best Practices → Updates).
- **Device didn't check in** — **Sync** and re-check **Device install status**.

Install status stuck "Pending"/"Failed" in Intune

- Give it a check-in cycle; then on the Mac inspect **Console.app** filtered for `MDM` / `AppInstall`, and the Intune app's **install error** detail. Re-run `MACVPP-Validation.ps1` after fixing.

Token / sync expiry (preventive)

- The **VPP token expires annually** and **blocks sync + installs** if lapsed. Schedule

`MACVPP-Discovery.ps1` (or `MACVPP-Remediation.ps1`) to run regularly; it flags anything within 30 days and can re-sync the token. Always renew with the **same managed Apple ID** and location.

Useful checkpoints

• **Intune:** Apps → macOS → (app) → **Device install status**; Apps → (VPP app) → **VPP licenses**; Tenant administration → **Connectors and tokens** → **Apple VPP tokens**.

• **Apple Business Manager: Apps** (license counts per location); **Locations** (the location exists); token owner (managed Apple ID).

• **On the Mac:** `/Applications` (app present); **System Settings** → **General** → **Device Management** (managed); **Console.app** for install logs.

• **Scripts:** re-run `MACVPP-Validation.ps1` after each change to confirm the gate.

7. Best Practices — macOS App Deployment & Apple VPP with Microsoft Intune

Field-proven guidance to keep the deployment reliable, correctly licensed, and low-maintenance.

Licensing model

• **Default to device licensing** for company-owned Macs. It installs silently, needs **no Apple ID or App Store sign-in**, and reclaims the license automatically when a device retires.

• **Reserve user licensing** for the rare app that genuinely requires it. Budget for the extra friction: users must be **invited**, **accept VPP terms**, and sign in to the App Store.

• **Right-size license counts.** Track **consumed vs. total** per app (`MACVPP-Report.ps1`) and keep headroom so a rollout wave never stalls **out of licenses**. Buy more in ABM under **Location A** before you expand, then **Sync**.

• **Reclaim on retire/unassign.** Device licenses return to the pool automatically; verify reclamation after large retirements so counts stay accurate.

Token & license-sync lifecycle (the #1 cause of outages)

• **The VPP token expires annually and a lapse stops sync and new installs.** Treat renewal as a scheduled task, not a surprise.

• **Renew with the same managed Apple ID** that owns the token, for the **same location**. Document that Apple ID and the location in your runbook (Requirements R4).

• **Schedule** `MACVPP-Discovery.ps1` to run regularly and alert on a token within 30 days of expiry or a stale last-sync time.

- **Keep one token per location, clearly named** (`VPP – Corporate (Location A)`) so licenses and

renewals are unambiguous. Acquire all apps you deploy under that one location.

App type selection

- **App Store → VPP.** If it's on the App Store, deploy it as a **VPP (App Store)** app so licensing is

tracked and installs are silent.

- **Own/vendor app with a signed `.pkg`/`.dmg` → managed LOB/DMG.** Get managed install, update, and

reporting.

- **Only fall back to unmanaged PKG** when the managed types can't run the installer (scripts, files

outside `/Applications`). Accept that it's install-once with no license/uninstall management.

- **Always sign (and notarize where required)** enterprise packages so Gatekeeper doesn't block them.

Assignment & targeting hygiene

- **Pilot first, expand in waves.** Assign to `PILOT – macOS App Deployment`, validate, then widen

with `MACVPP-Set-ScopeTargeting.ps1` (preview with `-WhatIf` before applying).

- **Prefer Required for hands-off delivery**; use **Available** (Company Portal) for opt-in apps.

- **Assign to device groups** for device-licensed VPP apps so licensing and targeting align.

- **Name and reuse objects** — the token, the pilot group, and each app — rather than creating

duplicates per wave.

Updates & versioning

- **Let store apps update automatically**; confirm managed Macs advance to new App Store versions.

- **For LOB, always bump version/build** on the new `.pkg` so Intune recognizes it as an update.

- **Test a new LOB package on the pilot** before broad release, exactly as you did the first install.

Operations & documentation

- **Keep a runbook** of the managed Apple ID, ABM location, token name, token expiry date, and the app

inventory (type + licensing model) — the values that, if lost, are painful to recover.

- **Report monthly** with `MACVPP-Report.ps1`: apps deployed, install coverage, and VPP

consumed-vs-total per app.

- **Re-validate after changes** with `MACVPP-Validation.ps1` so each change still meets the gate.

Security & posture

- **Deploy only acquired, approved apps**; VPP keeps the catalog controlled and licensed.
- **Combine with a macOS compliance/security baseline** (separate solutions) so app delivery lands on a

managed, compliant device.

- **Do not TLS-inspect Apple VPP/push endpoints** — it breaks license sync and install (Requirements R9).

8. Deliverables Summary — macOS App Deployment & Apple VPP with Microsoft Intune

Everything included in this solution package, and what each piece is for.

Documentation (this guide)

- **Executive Summary** — the business case and outcome in one page.
- **Solution Overview** — the macOS app-delivery model in Intune (VPP location tokens, device- vs. user-licensing, App Store vs. LOB/DMG/unmanaged PKG), how it works, benefits, and honest limits.
- **Requirements Guide** — every Apple/Intune prerequisite (ABM, a VPP location with acquired licenses, the VPP token, managed Macs, the pilot group, signed packages) with how to **verify** and how to **implement if missing**, plus the category-level Global Requirements.
- **Implementation Guide** — exact, ordered, click-by-click steps across four phases: acquire licenses and connect VPP, add and assign the App Store (VPP) app, add and assign the enterprise (LOB) app, then pilot/validate/expand — with an ongoing token-renewal section.
- **Testing & Validation** — eight tests with explicit pass/fail gates and an acceptance gate.
- **Troubleshooting** — top-down diagnosis for token/sync, install, and licensing failures.
- **Best Practices** — licensing model, the all-important token/license-sync lifecycle, app-type selection, targeting hygiene, updates, and operations.

PowerShell scripts (Microsoft Graph) — `MACVPP-Scripts.zip`

All scripts are tenant-side (run from an admin workstation with the Microsoft Graph PowerShell SDK); each has a header explaining **Purpose · How to run · Expected results**.

1. `MACVPP-Discovery.ps1` — tenant readiness: Apple VPP token status/expiry and last-sync time, synced App Store apps, and per-app VPP license totals. Flags a token expiring within 30 days. *Read-only.*

1. `MACVPP-Validation.ps1` — validates a specific Mac: managed macOS, the VPP app **installed and

device-licensed**, and the enterprise (LOB) app** installed** — with a clear PASS/FAIL verdict. *Read-only.*

1. `MACVPP-Report.ps1` — tenant-wide CSV report: per-app VPP **total vs. consumed** licenses, install counts, licensing model, and any app **out of licenses**. *Read-only.*

1. `MACVPP-Remediation.ps1` — safe remediation: re-sync the VPP token, surface tokens nearing expiry, and (opt-in) re-assign the store/LOB apps to a target group. **Defaults to** `-WhatIf`; changes require explicit confirmation.

1. `MACVPP-Set-ScopeTargeting.ps1` — builds/populates the target Entra group and (opt-in) retargets app assignments from pilot to a wave/all-devices group. **Defaults to** `-WhatIf`**.**

Hero image

• `MACVPP-hero.png` — the branded product image (1200×1200) used on the marketplace listing.

How it's delivered

A branded **Word** document and **PDF** (cover, table of contents, page numbers, and a scripts appendix), plus all five PowerShell scripts in a single **ZIP** — yours to keep and reuse across **unlimited devices**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MACVPP-Discovery.ps1

Purpose: Confirms the Intune tenant is ready to deploy and license Mac apps through Apple VPP: the Apple VPP token(s) status and expiry, last successful license sync, the count of synced App Store apps, and per-app VPP license totals. Flags any token expiring within 30 days or a stale last-sync time. Uses placeholders like your-location-token, not angle-bracket placeholders. HOW TO RUN Install the SDK once: Install-Module Microsoft.Graph -Scope CurrentUser Then: powershell -ExecutionPolicy Bypass -File .\MACVPP-Discovery.ps1 Required Graph scopes: DeviceManagementServiceConfig.Read.All, DeviceManagementApps.Read.All Reads only — makes no changes. EXPECTED RESULTS A PASS/FAIL line per check and an overall verdict. A READY tenant shows a Valid VPP token (expiry greater than 30 days), a recent last-sync, and at least one synced App Store app with VPP licenses. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param()

How to run: Install the SDK once: Install-Module Microsoft.Graph -Scope CurrentUser Then: powershell -ExecutionPolicy Bypass -File .\MACVPP-Discovery.ps1 Required Graph scopes: DeviceManagementServiceConfig.Read.All, DeviceManagementApps.Read.All Reads only — makes no changes. EXPECTED RESULTS A PASS/FAIL line per check and an overall verdict. A READY tenant shows a Valid VPP token (expiry greater than 30 days), a recent last-sync, and at least one synced App Store app with VPP licenses. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param()

MACVPP-Remediation.ps1

Purpose: Performs the common fixes that unblock VPP app delivery: re-sync the Apple VPP token with Apple, surface tokens nearing expiry, and (optionally) re-assign named App Store (VPP) and enterprise (LOB) apps to a target Entra group. Read-only by default (-WhatIf); changes require -Apply. Pass ids like your-entra-group-object-id, not angle-bracket placeholders. HOW TO RUN Install-Module Microsoft.Graph -Scope CurrentUser # once # Read-only audit / preview (default — makes NO changes): powershell -ExecutionPolicy Bypass -File .\MACVPP-Remediation.ps1 # Sync the VPP token(s) and re-assign named apps (explicit opt-in): powershell -ExecutionPolicy Bypass -File .\MACVPP-Remediation.ps1 -Apply -GroupId your-entra-group-object-id -AppIds id1,id2 Required Graph scopes: DeviceManagementServiceConfig.ReadWrite.All, DeviceManagementApps.ReadWrite.All EXPECTED RESULTS Without -Apply: an audit of token expiry/sync and what WOULD be done. With -Apply: a token sync is requested and the named apps are assigned Required to the group. .NOTES White-label — the repeatable way. Defaults to -WhatIf; changes require -Apply. Read-only against Microsoft Graph unless -Apply is supplied. #> [CmdletBinding(SupportsShouldProcess)] param([switch]$Apply, [string]$GroupId, [string[]]$AppIds)

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once # Read-only audit / preview (default — makes NO changes): powershell -ExecutionPolicy Bypass -File .\MACVPP-Remediation.ps1 # Sync the VPP token(s) and re-assign named apps (explicit opt-in): powershell -ExecutionPolicy Bypass -File .\MACVPP-Remediation.ps1 -Apply -GroupId your-entra-group-object-id -AppIds id1,id2 Required Graph scopes: DeviceManagementServiceConfig.ReadWrite.All, DeviceManagementApps.ReadWrite.All EXPECTED RESULTS Without -Apply: an audit of token expiry/sync and what WOULD be done. With

-Apply: a token sync is requested and the named apps are assigned Required to the group. .NOTES White-label — the repeatable way. Defaults to -WhatIf; changes require -Apply. Read-only against Microsoft Graph unless -Apply is supplied. #> [CmdletBinding(SupportsShouldProcess)] param([switch]$Apply, [string]$GroupId, [string[]]$AppIds)

MACVPP-Report.ps1

Purpose: Exports every macOS app in Intune with the fields that matter for this solution: app type (App Store/VPP vs. LOB/DMG/PKG), VPP licensing model, VPP total vs. consumed licenses, remaining headroom, and install counts. Flags any VPP app that is out of licenses. Use a path like C:\Temp\macvpp-report.csv, not an angle-bracket placeholder. HOW TO RUN Install-Module Microsoft.Graph -Scope CurrentUser # once powershell -ExecutionPolicy Bypass -File .\MACVPP-Report.ps1 powershell -ExecutionPolicy Bypass -File .\MACVPP-Report.ps1 -CsvPath C:\Temp\macvpp-report.csv Required Graph scopes: DeviceManagementApps.Read.All Reads only — makes no changes. EXPECTED RESULTS A summary (macOS apps, VPP apps, total vs. consumed licenses, apps out of licenses) plus a CSV with one row per app. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath = "$PSScriptRoot\macvpp-report.csv")

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once powershell -ExecutionPolicy Bypass -File .\MACVPP-Report.ps1 powershell -ExecutionPolicy Bypass -File .\MACVPP-Report.ps1 -CsvPath C:\Temp\macvpp-report.csv Required Graph scopes: DeviceManagementApps.Read.All Reads only — makes no changes. EXPECTED RESULTS A summary (macOS apps, VPP apps, total vs. consumed licenses, apps out of licenses) plus a CSV with one row per app. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath = "$PSScriptRoot\macvpp-report.csv")

MACVPP-Set-ScopeTargeting.ps1

Purpose: Creates (or reuses) the Entra security group used to target macOS app deployment, optionally adds members, and optionally retargets named App Store (VPP) / enterprise (LOB) app assignments from the pilot to a wave or all-devices group. Defaults to -WhatIf so nothing changes until you confirm. Pass names like PILOT - macOS App Deployment and ids like your-app-id, not angle-bracket placeholders. HOW TO RUN Install-Module Microsoft.Graph -Scope CurrentUser # once # Preview only (default -WhatIf — no change): powershell -ExecutionPolicy Bypass -File .\MACVPP-Set-ScopeTargeting.ps1 -GroupName 'PILOT - macOS App Deployment' # Create/populate the group and retarget apps (explicit opt-in): powershell -ExecutionPolicy Bypass -File .\MACVPP-Set-ScopeTargeting.ps1 -GroupName 'WAVE1 - macOS App Deployment' -MemberDeviceIds id1,id2 -AppIds appId1,appId2 -Confirm:$false Required Graph scopes: Group.ReadWrite.All, DeviceManagementApps.ReadWrite.All EXPECTED RESULTS With -WhatIf (default): prints the group it WOULD create/reuse and the assignments it WOULD set. Without -WhatIf: ensures the group exists, adds members, and assigns the named apps Required (device-licensed for VPP) to that group. .NOTES White-label — the repeatable way. Defaults to -WhatIf; changes require -Confirm:$false or approval at the prompt. Read-only against Microsoft Graph until a change is confirmed. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$GroupName, [string[]]$MemberDeviceIds, [string[]]$AppIds)

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once # Preview only (default -WhatIf — no change): powershell -ExecutionPolicy Bypass -File .\MACVPP-Set-ScopeTargeting.ps1 -GroupName 'PILOT - macOS App Deployment' # Create/populate the group and retarget apps (explicit opt-in):

powershell -ExecutionPolicy Bypass -File .\MACVPP-Set-ScopeTargeting.ps1 -GroupName 'WAVE1 - macOS App Deployment' -MemberDeviceIds id1,id2 -AppIds appId1,appId2 -Confirm:$false Required Graph scopes: Group.ReadWrite.All, DeviceManagementApps.ReadWrite.All EXPECTED RESULTS With -WhatIf (default): prints the group it WOULD create/reuse and the assignments it WOULD set. Without -WhatIf: ensures the group exists, adds members, and assigns the named apps Required (device-licensed for VPP) to that group. .NOTES White-label — the repeatable way. Defaults to -WhatIf; changes require -Confirm:$false or approval at the prompt. Read-only against Microsoft Graph until a change is confirmed. #> [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$GroupName, [string[]]$MemberDeviceIds, [string[]]$AppIds)

MACVPP-Validation.ps1

Purpose: For one macOS device (by serial number or device name), confirms it is a managed macOS device, that a target App Store (VPP) app is installed and device-licensed, and that a target enterprise (LOB) app is installed. Produces a clear PASS/FAIL verdict. Use app names like your-store-app and your-lob-app in the parameters, not angle-bracket placeholders. HOW TO RUN Install-Module Microsoft.Graph -Scope CurrentUser # once powershell -ExecutionPolicy Bypass -File .\MACVPP-Validation.ps1 -Serial C02XXXXXXXXX -StoreAppName 'your-store-app' -LobAppName 'your-lob-app' powershell -ExecutionPolicy Bypass -File .\MACVPP-Validation.ps1 -DeviceName 'Jane-MacBook' Required Graph scopes: DeviceManagementManagedDevices.Read.All, DeviceManagementApps.Read.All Reads only — makes no changes. EXPECTED RESULTS A per-check PASS/FAIL table and an overall verdict. A fully validated Mac passes every check; investigate any FAIL using the Troubleshooting guide. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$Serial, [string]$DeviceName, [string]$StoreAppName, [string]$LobAppName)

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once powershell -ExecutionPolicy Bypass -File .\MACVPP-Validation.ps1 -Serial C02XXXXXXXXX -StoreAppName 'your-store-app' -LobAppName 'your-lob-app' powershell -ExecutionPolicy Bypass -File .\MACVPP-Validation.ps1 -DeviceName 'Jane-MacBook' Required Graph scopes: DeviceManagementManagedDevices.Read.All, DeviceManagementApps.Read.All Reads only — makes no changes. EXPECTED RESULTS A per-check PASS/FAIL table and an overall verdict. A fully validated Mac passes every check; investigate any FAIL using the Troubleshooting guide. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$Serial, [string]$DeviceName, [string]$StoreAppName, [string]$LobAppName)